

Implementación de Listas Enlazadas en Java

Reporte de la tarea de Estructuras de Datos

Ejercicios: 2 (Cuenta letras), 4 (Inserción), 6 (Búsqueda) y 8 (Inversión)

Alumna: Yaneli Sánchez Cárdenas

Grupo: GTID244

Universidad Tecnológica del Norte de Guanajuato (UTNG)

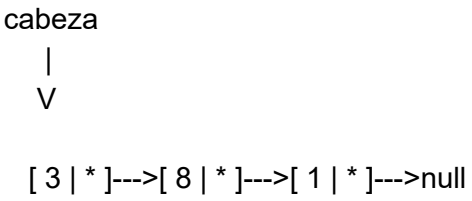
Fecha: 25 de octubre de 2026

Introducción

En esta tarea se resolvio cuatro ejercicios de omegaUp usando listas enlazadas simples hechas por mí. Una lista enlazada es una fila de cajitas llamadas nodos. Cada nodo guarda un dato y una flecha (el puntero siguiente) que dice cuál es la cajita que va después. El último nodo apunta a null, que significa "aquí se acaba".

En todos los ejercicios uso las mismas dos clases:

- Nodo: guarda el dato y el puntero siguiente.
- ListaSimple: guarda la cabeza (el primer nodo) y los métodos de cada ejercicio.
- Main: lee los datos con Scanner, usa la lista e imprime la respuesta.



Así se ve una lista con los datos 3, 8 y 1.

Ejercicio 2: Cuenta letras

Descripción del problema

El programa recibe una palabra (solo letras minúsculas, de 0 a 1000 letras) y tiene que decir cuántas veces aparece cada letra del alfabeto. Primero imprime una línea con 26 números (uno por cada letra, de la a a la z) y después una línea por cada letra que sí aparece, con el formato La letra x aparece n veces, por ejemplo, para programacion la primera línea es 2 0 1 0 0 0 1 0 1 0 0 0 1 1 2 1 0 2 0 0 0 0 0 0 0 0 y después salen líneas como La letra a aparece 2 veces.

¿Por qué una lista enlazada simple propia?

Usé una lista enlazada simple hecha desde cero, con mi clase Nodo y mi clase ListaSimple, sin usar LinkedList ni ArrayList de Java, porque la tarea pide implementar la estructura yo misma. Aquí cada letra de la palabra se guarda en un nodo (el nodo guarda un char). Así la lista puede crecer del tamaño que sea, y para contar una letra solo tengo que recorrer los nodos.

3. Cómo funciona el algoritmo y los punteros (siguiente**)**

Primero, Main lee la palabra y mete cada letra a la lista con agregar(). Después, para cada letra de la a a la z, llama a contarLetra() y guarda el resultado en un arreglo de 26 números.

- **agregar:** se crea un nodo nuevo. Si la lista está vacía, la cabeza apunta a él. Si no, uso un puntero auxiliar actual que empieza en la cabeza y avanza con actual = actual.siguiente hasta llegar al último nodo (el que tiene siguiente == null). Entonces hago actual.siguiente = nuevo y el nodo nuevo queda al final.
- **contarLetra:** pongo un contador en 0 y recorro la lista desde la cabeza. En cada nodo pregunto si el dato es la letra que busco; si sí, sumo 1. Luego avanzo con actual = actual.siguiente. Cuando actual es null, ya se acabó y regreso el contador.
- **Main:** imprime los 26 números en la primera línea y después una línea por cada letra cuyo contador sea mayor que 0.

Palabra "hola": cabeza ---> [h] ---> [o] ---> [l] ---> [a] --->
null

contarLetra('o'): recorre los 4 nodos y solo en [o] suma 1 -> 1
contarLetra('z'): recorre los 4 nodos y nunca suma -> 0

Casos de prueba

Caso	Entrada	Salida esperada	Qué comprueba
Normal	programacion	2 0 1 0 0 0 1 0 1 ... (26 números) La letra a aparece 2 veces ... (igual al ejemplo de omegaUp)	cuenta cada letra y solo imprime las que aparecen.
Caso	**Entrada**	**Salida esperada**	**que comprueba**
Normal 2	aaaaaccccceeegghxxzzz	5 0 4 0 3 0 2 1 0 ... 2 0 3 La letra a aparece 5 veces ...	Segundo ejemplo de omegaUp.
límite (vacía)	(línea vacía)	0 0 0 ... 0 (26 ceros) y ninguna línea más	La lista queda vacía y no truena.
grande	z repetida 1000 veces (el máximo)	0 0 ... 0 1000 La letra z aparece 1000 veces	Funciona con el máximo de nodos.

Las dos primeras salidas coinciden con los ejemplos del problema en omegaUp, las demás las comprobé corriendo el programa.

5. Capturas de pantalla

local



omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-26 21:43	java	100.00%	Terminada	Correcta	3.98 MB	0.35 s	

. Conclusiones

- Al principio entendí mal el problema: pensé que solo había que contar cuántos caracteres tenía lapalabra, y omegaUp me daba "Incorrecta". Al leer bien el enunciado vi que pide contar cuántas veces aparece cada letra, y cambié el método contarElementos por contarLetra.
- También me costó que omegaUp solo recibe un archivo: tuve que juntar Nodo, ListaSimple y Main en un solo Main.java (quitando el public de Nodo y ListaSimple). En mi carpeta local sí los dejé en tres archivos.
- La parte difícil de la lista fue llegar al final: hay que parar cuando actual.siguiente es null, no cuando actual es null, porque si no, no puedo enlazar el nodo nuevo.
- Los comentarios Javadoc al inicio de cada archivo (problema, autor, grupo, fecha y descripción)los hice con el formato /** ... */ y los repetí en todas las clases.

Ejercicio 4: Inserción en lista simple

1. Descripción del problema

Es el problema Inserción en Lista Enlazada de omegaUp. Se reciben números positivos, uno por línea, hasta que se acaben los datos. Hay que guardarlos en una lista y meter un nodo con -1 antes de cada número impar. El programa imprime cuántos nodos metió y después la lista, por ejemplo para 1, 2, 3 imprime 2 y luego -1 -> 1 -> 2 -> -1 -> 3 -> NULL.

2. ¿Por qué una lista enlazada simple propia?

Usé una lista enlazada simple hecha desde cero, con mi clase Nodo y mi clase ListaSimple, sin usar LinkedList ni ArrayList de Java, porque la tarea pide implementar la estructura yo misma. Este problema trata justo de meter nodos en medio de una lista, y eso solo se puede practicar si yo manejo los punteros. En una lista hecha con arreglos habría que mover todos los datos; en la mía solo cambio dos flechas.

3. Cómo funciona el algoritmo y los punteros (siguiente**)**

Main lee los números con insertar() (que agrega al final) hasta que no haya más. Después llama a insertarAntesDeImpares(), que recorre la lista con dos punteros: anterior (el nodo de atrás) y actual (el nodo que reviso). Si actual es impar (dato % 2 != 0), hago 3 cosas:

- Creo un nodo nuevo con -1.
- Hago que el nuevo apunte al impar: nuevo.siguiente = actual.
- Hago que el nodo de atrás apunte al nuevo: anterior.siguiente = nuevo. Si el impar era el primero de la lista (no hay nodo de atrás), entonces el nuevo pasa a ser la cabeza.

- Después avanzo: anterior = actual y actual = actual.siguiente. Así nunca vuelvo a revisar el -1 que acabo de meter.
- Cuento cada -1 que meto, y ese número es el que se imprime en la primera línea. La segundalínea la imprime mostrar(), que al final escribe NULL.

Lista 1 -> 2 -> 3
actual = [1], es impar: se mete un -1 antes. Queda -1 -> 1 -> 2 -> 3
actual = [2], es par: no se hace nada
actual = [3], es impar: se mete un -1 antes. Queda -1 -> 1 -> 2 -> -1 -> 3 -> NULL
Nodos insertados = 2

4. Casos de prueba

Caso	**Entrada**	**Salida esperada**	**Qué comprueba**
Normal (ejemplo omegaUp)	1 2 3	2 -1 -> 1 -> 2 -> -1 -> 3 -> NULL	Mete -1 antes de cada impar.
Límite (vacía)	(sin datos)	0 NULL	No hay nodos y no truena
Caso	**Entrada**	**Salida esperada**	**que comprueba**
límite (1 dato)	7	1 -1 -> 7 -> NULL	El -1 queda al inicio (cabeza).
sin impares	2 4	0 2 -> 4 -> NULL	No inserta nada.
Grande	los números del 1 al 2000	1000 (y una lista de 3000 nodos)	Hay 1000 impares y funciona.

El primer caso es el ejemplo del problema en omegaUp ,los demás los comprobé corriendo el programa.

5. Capturas de pantalla

Local



Omega up

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-26 21:56	java	100.00%	Terminada	Correcta	1.22 MB	0.68 s	

6. Conclusiones

- Lo más difícil fue meter un nodo en medio de la lista: hay que cambiar dos flechas y el orden importa. Si primero cambio anterior.siguiente, pierdo el nodo impar. Por eso primero hago que el nuevo apunte al impar y luego conecto el de atrás.
- También tuve que cuidar el caso del primer nodo, porque no tiene nodo de atrás y entonces secambia la cabeza.
- Al principio mi código no era lo que pedía omegaUp (formato de entrada y de salida distinto), locorregí leyendo el enunciado. Además omegaUp solo recibe un archivo, así que junté las clases en un solo Main.java.
- Las dificultades de siempre, las carpetas y los comentarios Javadoc, las resolví usando el mismoencabezado y la misma estructura en todos los archivos.

Ejercicio 6: Búsqueda en lista simple

1. Descripción del problema

Es el problema Cajas de dulces de omegaUp. Hay N cajas en un estante y cada una tiene cierta cantidad de dulces. Carlitos solo puede llevarse K cajas que estén juntas. Hay que buscar cuál es la suma más grande de dulces que puede llevarse. Se leen N y K y luego los N números. Con 10 4 y 3 9 9 1 9 10 8 7 2 16 la respuesta es 34 (las cajas 9, 10, 8 y 7).

2. ¿Por qué una lista enlazada simple propia?

Usé una lista enlazada simple hecha desde cero, con mi clase Nodo y mi clase ListaSimple, sin usar LinkedList ni ArrayList de Java, porque la tarea pide implementar la estructura yo misma. Aquí la lista guarda las cajas en el orden del estante, y para buscar la mejor suma tengo que recorrerla nodo por nodo, que es lo que se practica con el puntero siguiente.

3. Cómo funciona el algoritmo y los punteros (siguiente**)**

Main lee N y K, guarda las N cajas con insertar() y llama a buscarSumaMaxima(k). Ese método prueba empezar en cada nodo de la lista:

- Uso el puntero inicio para marcar dónde empieza cada grupo. Empieza en la cabeza.
- Desde inicio, con otro puntero actual, sumo K nodos seguidos avanzando con actual = actual.siguiente, y voy contando cuántos llevo.
- Si sí llegué a K nodos (no se acabó la lista antes) y la suma es mayor que la mejor que tenía, laguardo en maxima.
- Luego muevo inicio = inicio.siguiente y repito, hasta que inicio sea null. Al final regreso maxima.

Lista 9 -> 5 -> 2 -> 10 -> 3, con K = 2
inicio en 9: 9 + 5 = 14 (la mejor suma hasta ahora es 14)
inicio en 5: 5 + 2 = 7
inicio en 2: 2 + 10 = 12
inicio en 10: 10 + 3 = 13
inicio en 3: solo queda 1 nodo, no llega a K y no cuenta
Respuesta: 14

4. Casos de prueba

Caso	**Entrada**	**Salida esperada**	**Qué comprueba**
Normal 1 (omegaUp)	10 4 3 9 9 1 9 10 8 7 2 16	34	Ejemplo del problema.
Normal 2 (omegaUp)	5 2 9 5 2 10 3	14	Segundo ejemplo del problema.
Límite (K = N)	3 3 1 2 3	6	Solo hay un grupo posible: todas las cajas.
Caso	**Entrada**	**Salida esperada**	**Qué comprueba**
Límite (1 caja)	1 1 500	500	Lista de un solo nodo.
Grande	N = 1000, K = 300, todas las cajas con 500	150000	Funciona con 1000 nodos (el máximo).

Los dos primeros casos son los ejemplos del problema en omegaUp, los demás se comprobaron corriendo el programa.

5. Capturas de pantalla

local:



Omega uop



6. Conclusiones

- Mi primera versión de este ejercicio no resolvía el problema de omegaUp (yo había hecho otracosa). Al leer el enunciado entendí que había que sumar K nodos seguidos y quedarme con la mayor suma, y por eso cambié el método de búsqueda.
- Lo más importante fue cuidar el final de la lista: si empiezo cerca del final y ya no alcanzan Knodos, ese grupo no se cuenta.
- También fue difícil lo de siempre: la configuración de carpetas, los comentarios Javadoc, y queomegaUp solo recibe un archivo (junté Nodo, ListaSimple y Main en un solo Main.java).

Ejercicio 8: Inversión de lista simple

1. Descripción del problema

Es el problema Derdenio y sus Estructuras de Datos de omegaUp. Derdenio tiene dos listas de números ordenadas de menor a mayor, y quiere juntarlas en una sola que quede ordenada de mayor a menor. Se lee N, luego N números (primera lista) y luego otros N (segunda lista). Con 1 7 9 12 23 y 2 3 8 14 24 se imprime 24 23 14 12 9 8 7 3 2 1.

2. ¿Por qué una lista enlazada simple propia?

Usé una lista enlazada simple hecha desde cero, con mi clase Nodo y mi clase ListaSimple, sin usar LinkedList ni ArrayList de Java, porque la tarea pide implementar la estructura yo misma. Este problema necesita juntar dos listas y darle la vuelta a una lista, y las dos cosas se hacen cambiando punteros. Por eso necesito mi propia lista, y no una que ya traiga Java.

3. Cómo funciona el algoritmo y los punteros (siguiente**)**

Lo resolví en dos partes. Primero junto las dos listas (queda de menor a mayor) y después la invierto (queda de mayor a menor).

- Juntar (mezclarCon): uso un puntero p para la primera lista y otro q para la segunda. Comparo p.dato con q.dato, paso el más chico a la lista nueva y avanzo solo ese puntero (p = p.siguiente o q = q.siguiente). Cuando una lista se acaba, paso todo lo que le sobró a la otra.
- Invertir: uso tres punteros: anterior (empieza en null), actual (empieza en la cabeza) y siguienteNodo (un guardado temporal). En cada vuelta: guardo siguienteNodo = actual.siguiente, doy la vuelta a la flecha con actual.siguiente = anterior, y avanzo anterior = actual y actual = siguienteNodo.
- Al terminar, la nueva cabeza es el último nodo que procesé: cabeza = anterior. Después mostrar() imprime los números separados por un espacio.

Listas 1 -> 3 y 2 -> 4
Juntar:
1 contra 2: pasa el 1
3 contra 2: pasa el 2
3 contra 4: pasa el 3
Sobra el 4: pasa el 4
Resultado: 1 -> 2 -> 3 -> 4
Invertir (cada flecha se da la vuelta): 4 -> 3 -> 2 -> 1 -> null

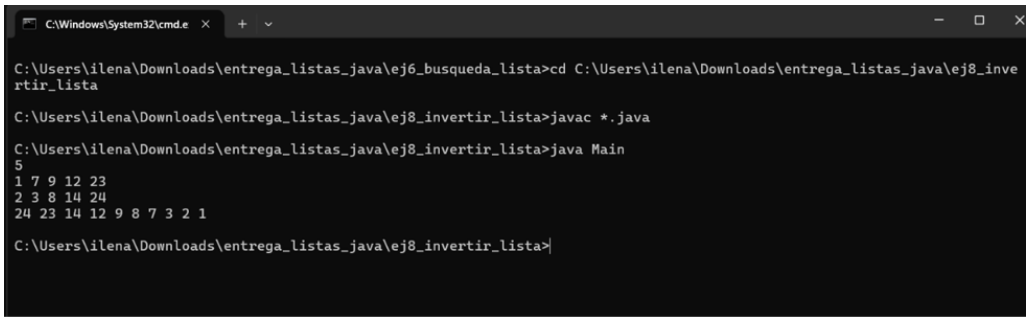
4. Casos de prueba

Caso	**Entrada**	**Salida esperada**	**Qué comprueba**
Normal (omegaUp)	5 1 7 9 12 23 2 3 8 14 24	24 23 14 12 9 8 7 3 2 1	Mezcla e invierte, como el ejemplo.
Caso	**Entrada**	**Salida esperada**	**Qué comprueba**
Límite (1 número)	1 5 3	5 3	Cada lista tiene un solo nodo.
Grande	N = 1000: impares del 1 al 1999 y pares del 2 al 2000	2000 números (del 2000 al 1)	Funciona con 2000 nodos.

El primer caso es el ejemplo del problema en omegaUp; los demásse comprobaron corriendo el programa.

5. Capturas de pantalla

Local



omegaup



6. Conclusiones

- Este fue el ejercicio más difícil porque hay que pensar en los punteros sin perder ningún nodo. Loque me ayudó fue dibujar las flechas en papel y seguir cada paso, como en el ejemplo de arriba.
- Si no guardara siguienteNodo antes de cambiar la flecha, perdería el resto de la lista.
- Mi primera versión solo invertía una lista, y omegaUp pide juntar dos listas ordenadas y dejarlasde mayor a menor, así que agregué el método de mezclar. También tuve que quitar el espacio que quedaba al final de la línea.
- Las dificultades con las carpetas, los comentarios Javadoc y el archivo único para omegaUp seresolvieron igual que en los ejercicios anteriores.

Conclusión

Con estos cuatro ejercicios vi cómo funciona una lista enlazada: no es un bloque de datos juntos, sino nodos conectados por punteros. Para casi todo hay que recorrer la lista con un puntero auxiliar que avanza con siguiente hasta llegar a null.

Lo que más me costó fue la configuración de carpetas y escribir los comentarios Javadoc, pero al hacer los ejercicios con la misma estructura (Nodo, ListaSimple y Main) se fue volviendo más fácil. El ejercicio de inversión fue el más complicado, pero dibujar los pasos me ayudó a entenderlo.